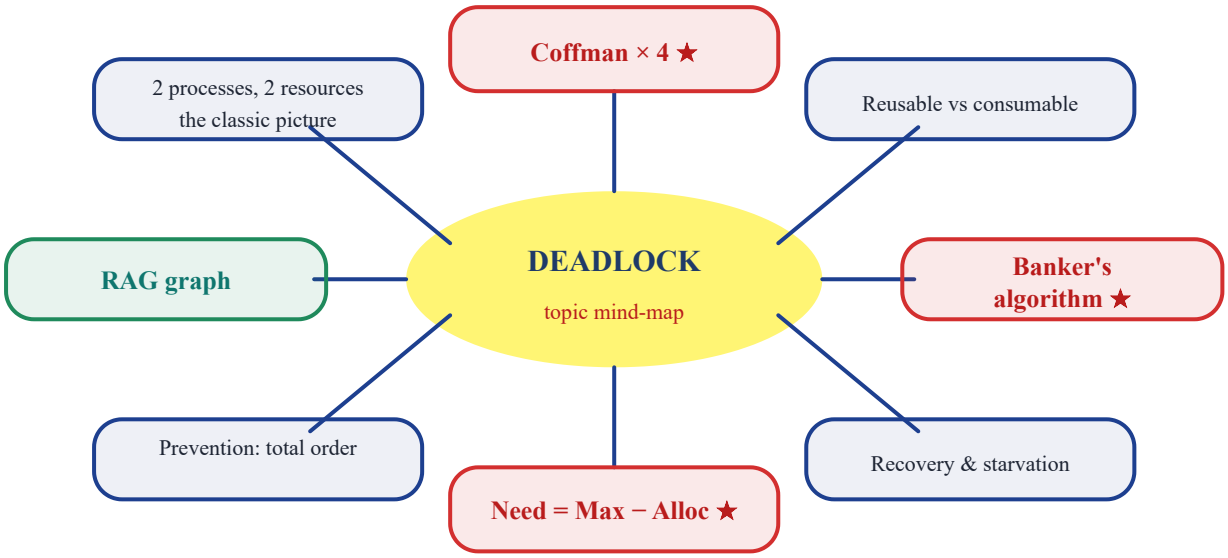
Deadlock

Coffman's four conditions · RAG cycles · Banker's algorithm · exam-ready in 11 sheets



↑ the whole topic on one page: red = high-yield, green = the one diagram every question is drawn from

Syllabus coverage

The syllabus line is "**Deadlock: resource allocation and deadlock, deadlock detection, prevention and recovery**", plus avoidance by the Banker's algorithm. Scheduling and IPC are in the Process lesson.

High-yield list

① Coffman × 4: M, H, N, C
② Claim R→P · request P→R
③ Cycle = deadlock only if 1 inst
④ Need = Max − Allocation
⑤ Work = Available, Work += Alloc
⑥ Safe = every Finish[i] true
⑦ Unsafe ≠ deadlocked
⑧ Break circular wait: total order
⑨ Recover: terminate / preempt
⑩ Aging → starvation, not deadlock

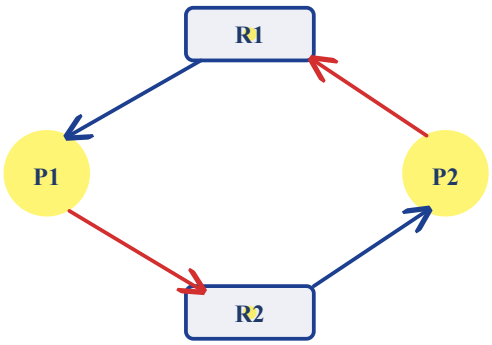
Legend: **yellow** = key term · **red circle** = must-memorise fact · **purple box** = memory trick · **red box** = exam trap · **green** = worked example

Contents

✓ What a deadlock is … p. 2
✓ Coffman's four conditions … p. 3
✓ Resource types & the RAG … p. 4
✓ Cycle ⇒ deadlock? … p. 5
✓ Prevention … p. 6
✓ Avoidance: safe states … p. 7
✓ Banker's algorithms … p. 8
✓ Detection & recovery … p. 9
✓ Night-before p. 10 · Practice p. 11

1 What Is a Deadlock?

Definition: A **deadlock** is a set of processes such that **every** member is blocked and waiting for an event that only another blocked member of the same set can cause or allow. Nothing can proceed, nothing outside can help, and **the situation can never resolve itself**. One process waiting for a busy device is **not** a deadlock — that is ordinary blocking.



blue = claim (allocated) · red = request (waiting)

R1 and R2 have **one instance each**: P1 holds R1 and wants R2; P2 holds R2 and wants R1

The classic two-process, two-resource example. A printer (R1) and a disk drive (R2), one instance each, exclusive use.

1. **P1** takes **R1**, the printer, for its whole print job.
2. **P2** finishes its disk work and asks for **R1**; it blocks.
3. P1 now needs **R2** for its spool file; R2 is free, so P1 takes it.
4. P1 finally wants to **print** — the printer is held by P2, which waits for P1. **Both blocked for ever.**

Term	Meaning	Example here
Request / wait for	the process asks for an instance it does not have	P1 waits for R1
Hold / allocate	the process has it and has not released it	P2 holds R2
Deadlock set	the whole set of mutually blocking processes	{P1, P2}

Memory trick: "Deadlock needs **at least two**, and needs **all four**." One process can only be starved or blocked; a deadlock always involves a **set** and a **cycle**. All four Coffman conditions must hold at once — knock out **one** and it cannot form.

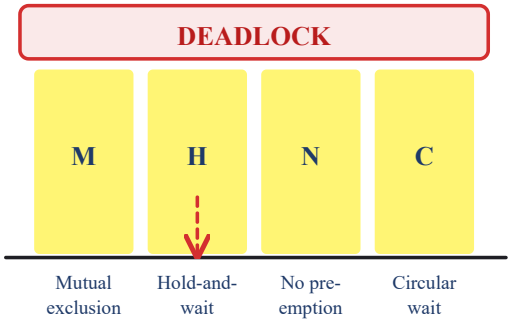
Exam trap: (i) A process waiting for a **busy device** is **not** a deadlock. (ii) A **cycle is not always a deadlock** — sheet 5. (iii) Deadlock ≠ starvation: in starvation the system keeps working and one process is cheated; in a deadlock **nothing** progresses.

Worked example ✍️ read the picture in words: R1 = one printer, R2 = one disk drive. **(a)** P1 → R1 alone: **no** deadlock — P1 is merely waiting, nothing holds R1. **(b)** P1 → R1 and P2 → R2 only: **no** — both instances are free, so one process gets one and the other waits harmlessly. **(c)** R1 → P1, P1 → R2, R2 → P2, P2 → R1? **Yes** — a closed chain in which every holder waits for the next, and that chain **is** the deadlock. **(d)** Add R2 → P3 with no request from P3: still a deadlock? **Yes** — the {P1, P2} cycle stands on its own.

Where you meet it in practice: Two database transactions each updating account A then B in opposite orders; a thread holding a mutex while it blocks on a socket read. The moral is always the same: **fix the order in which things are taken**.

2 Coffman's Four Necessary Conditions

Definition: Proposed by **Edward Coffman (1971)**: a deadlock is possible **if and only if all four** of these hold **at the same time**. They are **necessary conditions**, not a definition — none alone causes a deadlock, and denying any **one** of them makes deadlock impossible.



the roof falls only when **all four** pillars stand — pull one out (dashed) and nothing traps

Condition	Exactly what it says
Mutual exclusion	**at least one** resource is non-sharable: one process at a time
Hold and wait	a process **holds at least one** resource while **waiting** for more
No preemption	a resource comes back only when its holder releases it voluntarily
Circular wait	a closed chain: P0 waits for what P1 holds, …, Pn waits for what P0 holds

Memory trick: **MHNC** — "Mutual, Hold-and-wait, No-preemption, Circular". A room with **four doors**: you are trapped only when all four are shut. On MCQs, "which is **NOT** a condition?" → the answer is **aging** (it fights starvation), or priority inversion, or thrashing.

Exam trap: (i) Only **at least one** resource must be non-sharable. (ii) If every resource is sharable there is no deadlock. (iii) "Most practical prevention" = breaking **circular wait** by a total order. (iv) The conditions are **necessary**, not sufficient: some states satisfy all four and still complete.

Worked example ✍️ which conditions can you break here?: A word processor holds the **print-queue lock**, the **spell dictionary** and a **temporary file** while the user prints, and writes the dictionary to disk first. Its **clipboard** is sharable. **(a)** Mutual exclusion — **cannot** break it for the print queue or a temp file open for writing; **can** break it for the dictionary by making it read-only and shared. **(b)** Hold and wait — breakable: **release the dictionary before taking the print queue**, or ask for everything up front. **(c)** No preemption — breakable for the dictionary (page it out, reload later) but **not** for half-printed pages on paper. **(d)** Circular wait — breakable by ordering the locks: **clipboard < dictionary < temp file < print queue**.

The four strategies in one line each:

Prevention = deny one condition a priori

Avoidance = keep every state safe

Detection = look for one after it formed

Recovery = break it once found

3 Resource Types & the Resource-Allocation Graph

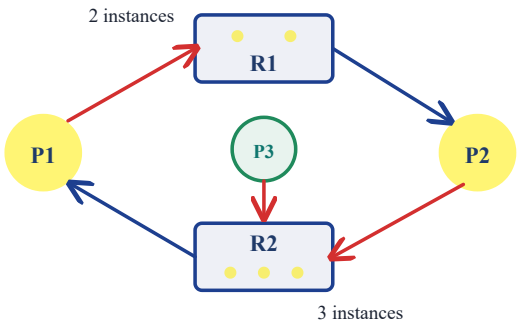
Definition: A **resource-allocation graph (RAG)** is a directed bipartite graph. **Process nodes** are **circles**, **resource nodes** are **squares**, and a square carries one **dot per instance** (no dot left = none free). Every edge joins a process to a resource, never two processes or two resources.

Edge	Direction	It means
Claim (allocation) edge	**Resource → Process**	that instance is **allocated** to the process — it holds it and has not released it
Request edge	**Process → Resource**	the process has asked for an instance of that type and is **waiting** for it

Classification	Meaning & examples
Reusable (serialisable)	used, released, available again — **never depleted**: CPU, memory, printer, disk, a mutex
Consumable	used **up**, once gone gone: a printed page, a spool job, a queue message
Sharable	many processes at the same instant: read-only file, code, shared read-only memory
Exclusive (non-sharable)	one process at a time: printer, tape drive, file open for writing, a lock

Memory trick: "**R**eusable is **R**eturned; **C**onsumable is **C**onsumed." A consumable resource can **never** be preempted — half a printed page cannot be un-printed.

Exam trap: Arrow direction is the most-tested detail here. **R → P is the claim edge (allocated)**; **P → R is the request edge (waiting)**. A process holding **every** instance it asked for has **no** outgoing request edge — then it is **not** deadlocked.



blue = claim (allocated), **red = request** (waiting): a cycle P1 → R1 → P2 → R2 → P1 exists, yet **P3** can still run — R2 has a spare instance, so this is **not** a deadlock

4 Does a Cycle Mean Deadlock?

The rule in one sentence: A deadlock always produces a cycle, so a cycle is **necessary**. But it is **sufficient as well only when every resource type has exactly one instance**. With several instances a cycle is necessary **but not sufficient**: a process outside it may still finish and hand back an instance that breaks the cycle.

Situation	Cycle ⇒ deadlock?
Every type has **1 instance**	**YES — always**; just look for a cycle
Any type has **> 1 instance**	**NO — maybe**; run the reduction

Worked example ✍ reduce sheet 4's graph: R1 = 2 instances, R2 = 3 instances. P1 holds 1 × R1, wants R2; P2 holds 2 × R2, wants R1; P3 holds nothing, wants R2. **Free** R1 = 2 − 1 = **1**; free R2 = 3 − 2 = **1**.

1. P3's R2 request is paid (1 ≥ 1) → **P3 finishes** and frees it again.
2. P1's R2 request is paid. P1 finishes and returns its R1 → free R1 = 0 + 1 = **1**, free R2 = 0.
3. P2's R1 request is paid (1 ≥ 1). P2 finishes.

Reduces away ⇒ a cycle but NO deadlock.

The reduction test, formally:

1. Count the free instances: **free[j] = total[j] − allocated[j]**.
2. Repeat: find a process P_i whose outstanding request can be paid out of the free counts. Assume P_i finishes and **returns every instance it holds**: free[j] += Allocation[i][j]; mark P_i finished.
3. All marked → **no deadlock**. Some unmarked and still waiting → **deadlocked**.

Exam trap: Never write "cycle = deadlock" without the qualifier. **Deadlock ⇒ cycle** always; **cycle ⇒ deadlock** only for single-instance resources. **High-yield**: for a single-instance graph "a deadlock **definitely** exists"; for a multiple-instance graph "a deadlock **may** exist".

Memory trick: "**One dot, one answer.**" If every square holds a single dot the cycle **is** the proof. If a square holds two or more dots, someone may still pay for somebody's request — so **peel the graph** and look at what is left.

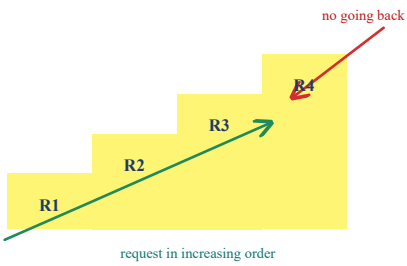
Worked example ✍ with 1 instance of R1: R1 = **1** instance (held by P1), R2 = 3 instances (2 held by P2). Free R1 = 0, free R2 = 1. P3 finishes, then both counts are 0 and nobody can move →
the cycle P1 → R1 → P2 → R2 → P1 is a real deadlock
.

IT link: Lock-ordering analysers run this exact reduction over the lock graph of a running JVM. A graph that will not peel has found a deadlocked thread.

5 Prevention: Break One of the Four

Definition: **Prevention** imposes a rule that keeps **at least one** Coffman condition permanently false, **before** any process runs. Nothing is detected and nothing is recovered — the deadlock simply cannot be built.

Condition denied	The rule	What it costs
Mutual exclusion	make resources **sharable**: read-only files or a **spooler**	a printer is inherently exclusive → **not always possible**
Hold and wait	demand **all** at once, or **release everything held** before asking more	low utilisation; losing every race introduces **starvation**
No preemption	if a request cannot be granted, **take back** what it holds	works for **CPU, memory, files**; **impossible for a printer**
Circular wait ★	a **total order**; a request must be for a **higher** number	wrong-order requests start again; nothing is wasted



each type gets a fixed number **a priori**

Worked example ✍ enforcing the order: Order: R1 (tape) < R2 (disk) < R3 (printer) < R4 (plotter). **(a)** P wants R3 then R2. R2 is lower → **refused**; P must finish with R3, release it, then ask again. **(b)** P wants R1 then R4. Both ascending → **granted**. **(c)** Can a cycle form? Each process holds only lower-numbered resources than the one it waits for, so the wait chain strictly **decreases** → **circular wait is impossible**. **(d)** P holds R4 and asks for a second instance of R4 → **refused**: never wait for another instance of a type you hold.

Exam trap: (i) Prevention asks for **all** resources at once; avoidance asks whether a request keeps the system **safe**. (ii) A spooler makes a printer sharable *in effect*, but the spool file is exclusive. (iii) Preemption of a **consumable** resource is impossible.

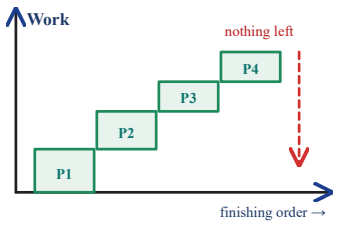
Memory trick: Denying circular wait is **the usual practical answer**, because the total order is fixed by the designer **a priori** — the OS needs **no knowledge of future claims at all**. Avoidance instead demands that every process declare its maximum claim first.

This is what real systems do: Oracle, PostgreSQL and every careful codebase impose a **lock ordering**; Java code that touches two locks always takes them in the same global order. Two threads that grab A then B and B then A will hang the service — the textbook cycle, in production.

6 Avoidance: Safe States

Definition: **Avoidance** with the **Banker's algorithm** (Dijkstra, 1965) lets processes request what they like, but **checks every request in advance** and grants it only if the resulting state keeps the possibility of every process finishing. The price: each process must declare **in advance** the **maximum number of instances of each type it may ever need**.

Safe state: There exists an order $\langle P_1, \ldots, P_n \rangle$ — a **safe sequence** — in which each process can obtain all its **remaining** need from what is free at that moment, then finish and hand everything back. No such order ⇒ **unsafe**.



each step is paid for by the work free at that moment

Data structure	Size	Meaning
Available[j]	m	free units of type j now
Max[i][j]	n×m	i's declared maximum claim
Allocation[i][j]	n×m	units of type j i holds
Need[i][j]	n×m	**Need = Max − Allocation**

Exam trap: **Unsafe ≠ deadlocked.** An unsafe state is one in which the **declared** maxima can no longer be met in any order; the processes may still finish if their real needs are smaller. A **safe state is guaranteed deadlock-free**; unsafe is only a warning. Also: **Need = Max − Allocation**, never Max − Available.

Memory trick: "**N**eed is what I still have to **ask** for." So **Work** is only compared with **Need**, while **Allocation** is what gets **added back** when a process finishes. Work never shrinks — it only grows, which is why the loop must end.

The four-word distinction:

Prevention: a rule imposed *before* — "you may never do X"

Avoidance: a decision at request time — "yes, this keeps us safe"

Detection: a periodic check — "is one there now?"

Recovery: action after the fact — "kill or preempt a victim"

Worked example ✍ safe or unsafe: Four identical tape drives, **Available = 2**. P1 holds 2 and needs 0 more; P2 holds 0 and needs 1; P3 holds 0 and needs 4. Need[P1] = 0 ≤ 2 ✓ → P1 finishes and returns its 2 drives: Work = 4. Need[P2] = 1 ≤ 4 ✓. Need[P3] = 4 ≤ 4 ✓. **Safe ⟨P1, P2, P3⟩.** Now Available = 1: P1 and P2 finish (Work = 3), but P3 needs 4 → **unsafe**.

Why avoidance is taught but rarely used: Every process must **declare a maximum in advance**, and real programs rarely know their own peak — so the OS over-reserves and utilisation falls. The safety check is cheap; the **prediction** is the expensive part.

Exam trap: The ≤ in **Need[i] ≤ Work** is **component-wise**: Need = (7, 5, 3) vs Work = (10, 4, 5) fails because 5 > 4, yet the totals 15 < 19 look fine. Never compare vectors by their sum.

7 The Banker's Algorithms, Worked Through

Algorithm 1 — safety (is this state safe?):

1. **Work = Available**; **Finish[i] = false** for all i.
2. Find any i with **Finish[i] = false** and **Need[i] ≤ Work** (component-wise).
3. Then **Work = Work + Allocation[i]**; set **Finish[i] = true**; back to step 2.
4. All Finish true → **SAFE**, and that order **is** the safe sequence. Some i stays false → **UNSAFE**.

Algorithm 2 — request (may it be granted?):

1. If **Request[i] > Need[i]** → **error**: the declared maximum was exceeded.
2. If **Request[i] > Available** → **wait**: not enough free now.
3. **Pretend** to grant: Available −= Request; Allocation[i] += Request; Need[i] −= Request.
4. Run the safety algorithm on the pretended state. **Safe → grant for real**; unsafe → **undo**, tell it to wait.

Process	Allocation			Maximum			Need = Max − Alloc		
	A	B	C	A	B	C	A	B	C
P0	0	1	0	7	5	3	**7**	**4**	**3**
P1	2	0	0	3	2	2	**1**	**2**	**2**
P2	3	0	2	9	0	2	**6**	**0**	**0**
P3	2	1	1	2	2	2	**0**	**1**	**1**
P4	0	0	2	4	3	3	**4**	**3**	**1**
Available	**3**	**3**	**2**	totals: A = 10, B = 5, C = 7 instances					

Step	Process	Need	Work in	Work out
1	P1	1 2 2	3 3 2 ✓	**5 3 2**
2	P3	0 1 1	5 3 2 ✓	**7 4 3**
3	P0	7 4 3	7 4 3 ✓	**7 5 3**
4	P2	6 0 0	7 5 3 ✓	**10 5 5**
5	P4	4 3 1	10 5 5 ✓	**10 5 7**



the Work vector of that walk — it only ever grows

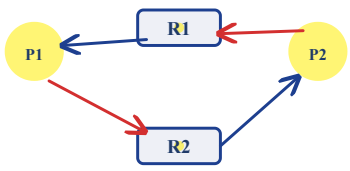
Result ✍ and now a request: Every Finish[i] became true, so the state is **SAFE** with safe sequence ⟨P1, P3, P0, P2, P4⟩. Check: the final Work (10, 5, 7) = Available + the whole Allocation column sum, (3+7, 3+2, 2+5) = (10, 5, 7) ✓. Now **Request[P1] = (1, 0, 2)**: ≤ Need[P1] ✓ and ≤ Available ✓. Pretend: Available = (2, 3, 0), Need[P1] = (0, 2, 0); the walk P1 → (5, 3, 2), P3 → (7, 4, 3), P0 → (7, 5, 3), P2 → (10, 5, 5), P4 → (10, 5, 7) finishes all five → **GRANT**. But **Request[P4] = (3, 3, 0)** gives Available = (0, 0, 2), Need[P4] = (1, 0, 1): no process fits → **REFUSE**.

Exam trap: Step 3 hands a process its whole remaining **Need**, but Work is increased by its **entire Allocation** — not its Need, not its Max. P3 gets (0, 1, 1) yet contributes (2, 1, 1), because P3 already held (2, 1, 1).

Memory trick: **Need** is the bill, **Allocation** is the wallet: the banker pays the bill and the whole wallet comes back. Max never enters the loop — it was only used once, to build Need.

8 Detection, Recovery & Starvation

Definition: **Detection** assumes the OS lets deadlocks happen, then **periodically** runs a check. It keeps the RAG plus **Request[i][j]** (what each process still waits for) and **Allocation[i][j]** — the same bookkeeping as the Banker's algorithm, used once instead of on every request.



single-instance detection: the RAG **is** the algorithm

Case	Detection method
Single instance of every type	the RAG with one dot per square; **any cycle = deadlock**; no extra matrix
Multiple instances	the **reduction** test of sheet 5: pay off any process the free units can satisfy, adding back its Allocation
Cost	$O(n^2)$ or worse — which is why real systems do not do this

Recovery 1 — process termination: Kill **all** deadlocked processes, or kill them **one at a time in a chosen order** until the cycle breaks; a killed process releases everything it held. Usual order, cheapest damage first: **least CPU time used so far → fewest resources held → fewest important resources held → longest CPU time still to run**.

Recovery 2 — resource preemption: **Take resources away** from a victim and hand them to a blocked process. The victim can only be restarted by **rollback** — restoring it to a previous safe state or checkpoint — because it may have already **partly used** the resource taken.
Guard against starvation: count rollbacks before preempting the same process again.

Point	Deadlock	Starvation
What happens	a **set** of processes waits on itself; nothing in it can move	one process is **indefinitely denied** what it needs
Who is affected	**every process in the cycle**	only the unlucky process; everyone else progresses
Cure	termination, or preemption + rollback	**aging**, fair queues

Memory trick: "**Deadlock stops the world, starvation picks a victim.**" Deadlock = a **set** stuck together; starvation = a **single** process shut out, with every state changing constantly. And **aging never fixes a deadlock** — nothing is progressing, so no priority is ever re-evaluated.

The practical bottom line: Real operating systems and databases **largely avoid deadlock by design rather than detect it**: they use **ordered locking** (a global total order on locks) and **no hold-and-wait** (one lock at a time, or release before requesting the next). Where a cycle is still possible they set a **timeout** — a lock timer, a `tryLock` with a deadline, a watchdog that dumps stacks and aborts.

Night-Before-the-Exam Sheet

Fact sheet:

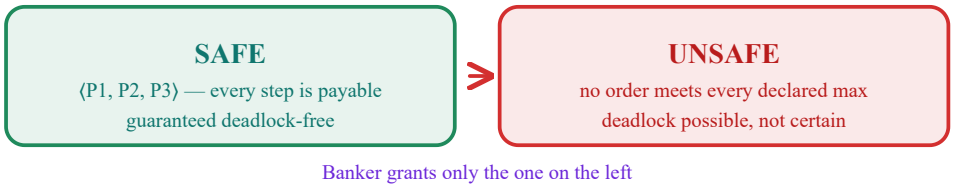
Coffman (1971): **M**utual exclusion	**H**old and wait
No preemption	**C**ircular wait
Claim edge **R** → **P** (allocated)	Request edge **P** → **R**
1 instance ⇒ cycle = deadlock	many ⇒ run the reduction
Need = Max − Allocation	**Work = Available**
Work += Allocation[i]	all Finish true ⇒ **safe**
Request > Need ⇒ error	Request > Available ⇒ wait
Unsafe ≠ deadlocked	total order set **a priori**
Reusable vs consumable	Terminate: least CPU used
Preempt ⇒ **rollback**	Aging ⇒ starvation only

Top 12 one-liners for MCQs

1. Needs **all four** conditions
2. Only **at least one** resource exclusive
3. Deadlock ⇒ cycle; cycle ⇒ deadlock **only** if single-instance
4. Claim **R** → **P**, request **P** → **R**
5. Circle = process, square = resource, dots = instances
6. Prevention: **break circular wait**
7. Banker's = **avoidance** (Dijkstra)
8. **Need = Max − Allocation**
9. Work = Work + Allocation[i]
10. Unsafe is only a **warning**
11. Recover: **terminate** or **preempt + rollback**
12. Preemption **impossible** for a printer

Top traps: cycle = deadlock **without** the single-instance qualifier · Need ≠ Max − Available · waiting for a busy device is **not** a deadlock · "at least one" resource, not all · unsafe ≠ deadlocked · aging fixes starvation, never deadlock · a process holding every instance it asked for has **no** request edge and is **not** deadlocked.

All tricks together: "Deadlock needs at least two, and needs all four" · "a room with four doors — deny one and you are free" · MHNC · "one dot, one answer" · "R is returned, C is consumed" · "the arrow leaves the holder and enters the wanted" · "the order is fixed a priori" · "Need is what I still have to ask for; Work only grows" · "deadlock stops the world, starvation picks a victim".



Banker grants only the one on the left

the whole avoidance idea in two boxes

✓ Practice MCQs (exam pattern, 10 marks)

1. A deadlock occurs only when:
(a) all four hold (b) any three hold
(c) circ. wait only (d) no mutual excl.

2. One instance of each resource:
(a) maybe, not sure (b) always deadlocks
(c) a safe state (d) only starvation

3. A claim (allocation) edge:
(a) res → holder (b) proc → wanted
(c) proc ↔ proc (d) type ↔ type

4. Usual practical prevention:
(a) preemption (b) all at once
(c) a total order (d) all sharable

5. The Banker's algorithm is for:
(a) detection (b) prevention
(c) avoidance (d) recovery

6. Need[i][j] equals:
(a) Max + Alloc (b) Avail − Alloc
(c) Max − Avail (d) Max − Alloc

7. Work starts at Available, then:
(a) + Max[i] (b) − Alloc[i]
(c) + Need[i] (d) + Alloc[i]

8. After preemption the victim:
(a) is rolled back (b) resumes as was
(c) is killed (d) gets new locks

9. NOT one of the four conditions:
(a) hold and wait (b) aging
(c) circ. wait (d) mutual excl.

10. A total order needs in advance:
(a) max claims (b) not even that
(c) CPU burst size (d) disk geometry

Answer key → 1-a · 2-b · 3-a · 4-c · 5-c · 6-d · 7-d · 8-a · 9-b · 10-b

Written question: State the four Coffman conditions and how prevention denies each. Draw the RAG of two deadlocked processes and say what a cycle proves with one instance per type. For P0–P4 as tabulated with Available = (3, 3, 2), find the safe sequence and decide whether Request[P1] = (1, 0, 2) may be granted. 10

1. **Conditions** — (i) **mutual exclusion**: at least one resource non-sharable; (ii) **hold and wait**: holds one, waits for more; (iii) **no preemption**: voluntary release only; (iv) **circular wait**: a closed chain P0 → … → Pn → P0. Denied by: sharable resources or a spooler; all-at-once or release-before-request; preemption of CPU/memory/files (never a printer); a total order. 3
2. **The graph** — P1, P2 circles; R1, R2 squares with one dot each; claim R1 → P1, R2 → P2; request P1 → R2, P2 → R1. One instance per type ⇒ a cycle is **necessary and sufficient**. 3
3. **The walk** — Need = 7 4 3 / 1 2 2 / 6 0 0 / 0 1 1 / 4 3 1. Work (3, 3, 2) → P1 (5, 3, 2) → P3 (7, 4, 3) → P0 (7, 5, 3) → P2 (10, 5, 5) → P4 (10, 5, 7) ⇒ **safe**, ⟨P1, P3, P0, P2, P4⟩. 3

4 **The request** — (1, 0, 2) ≤ Need[P1] = (1, 2, 2) ✓ and ≤ Available ✓. Pretended Available = (2, 3, 0), Need[P1] = (0, 2, 0); the safety run finishes all five ⇒ **grant**. 1

Before the exam ✓

- ✓ Can name the four Coffman conditions
- ✓ Can draw an RAG, claim vs request
- ✓ Can run the safety algorithm on 5 × 3
- ✓ Can tell starvation from deadlock